Day 129: Variational Autoencoders: the Reparameterization Trick From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 4 | Complete study guide

Watch: The Reparameterization Trick (ML & DL Explained) | The Reparameterisation Trick | Variational Inference (Machine Learning and AI Academy)

Script: `day129_vae_reparameterization.py` · every number in this guide is copied by program from `day129_run_output.txt`, the real output of that script.

A variational autoencoder maps each image to a small Gaussian cloud of codes and draws from it. To train through that random draw, the reparameterization trick writes it as $z = \mu + \sigma\varepsilon$. Everything is built from scratch and checked: the KL formula against a million-draw estimate, a gradient check through a whole VAE (3.1e-09), and the trick against the alternative score-function estimator. Both estimators are right on average, but the score function's estimates spread 7 times wider (14.85 against 1.99); used to learn a Gaussian, it blew up in 3 of 20 runs at the step size where the trick never did.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

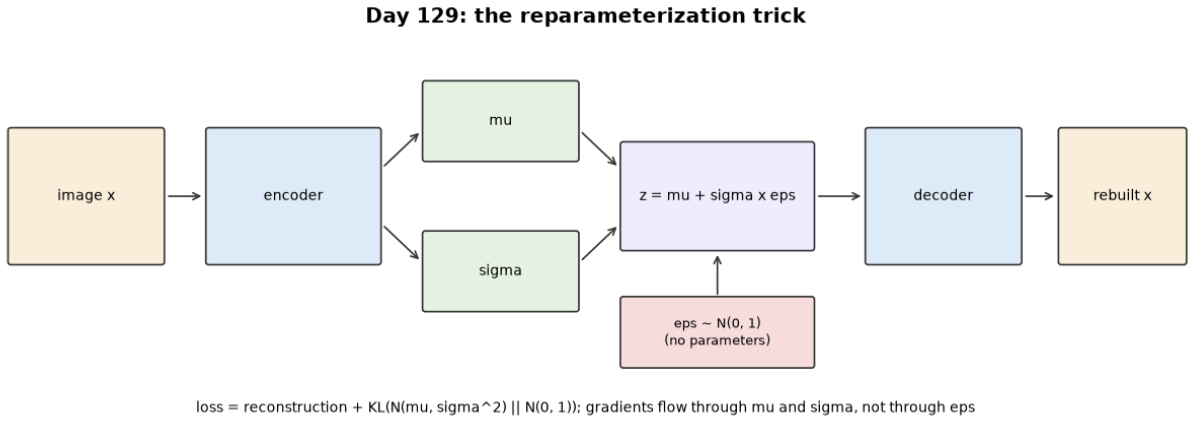
A **variational autoencoder** (VAE) has an encoder that outputs, for each image, a **mean** μ and a **spread** σ for every code number. A code z is **drawn** from $N(\mu, \sigma^2)$ and decoded. The loss is the reconstruction error plus the **KL divergence** from $N(\mu, \sigma^2)$ to $N(0, 1)$, which pulls every image's cloud towards the same standard normal. The total is the negative **ELBO** (evidence lower bound).

The **reparameterization trick**: draw ε from $N(0, 1)$ and set $z = \mu + \sigma\varepsilon$. The randomness sits in ε, which has no parameters, so z is an ordinary function of μ and σ and backpropagation goes straight through.

REAL-WORLD SCENARIO

A dart thrower aiming at a point. 'Where the dart lands' is random, but it is 'where you aim' plus 'your usual wobble'. To improve, you adjust the aim and steady the hand; the wobble itself is not yours to change.

SYSTEM DIAGRAM



The encoder gives μ and σ; ε brings the randomness; gradients flow through μ and σ.

Today's part for the week's build

`day129_exercise.py` is the trick, the KL term, the VAE loss and both estimators, checked by hand. Its real output:

ACTUAL OUTPUT

```
KL(N(1, 4) || N(0, 1)): 1.3069; by hand 0.5 (4 + 1 - 1 - log 4) = 1.3069
200,000 draws with mu 2, sigma 3: mean 2.010, spread 2.994 (by hand 2 and 3)
d E[z^2] / d(mu, sigma) at (1, 2): exact (2, 4); score function (2.02, 3.99), spread (8.6, 18.6);
    reparameterized (2.01, 4.01), spread (4.0, 6.0)
gradient check through a small VAE, eps fixed: largest gap 2.0e-09
```

Data: one-dimensional Gaussians where every answer is known in closed form, and MNIST (cached by Day 126) for a real VAE. Day 126's network parts are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: one draw, and one KL divergence

DEFINITION — start here, no prior knowledge assumed

$\mu = 1$, $\sigma = 2$ (log-variance log 4), $\varepsilon = 0.5$.

REAL-WORLD SCENARIO

Aiming at 1 with a wobble of 2; this throw wobbled half a unit.

MATHEMATICS

$$z = \mu + \sigma\varepsilon; \quad KL(N(\mu, \sigma^2) \,||\, N(0, 1)) = \tfrac{1}{2}(\sigma^2 + \mu^2 - 1 - \log \sigma^2)$$

IMPLEMENTATION

```
def reparam(mu, logvar, rng):
    eps = rng.standard_normal(mu.shape).astype(mu.dtype)
    return mu + np.exp(0.5 * logvar) * eps, eps
def kl_normal(mu, logvar):
    return 0.5 * (np.exp(logvar) + mu ** 2 - 1 - logvar).sum(1)
```

ACTUAL OUTPUT

```
mu 1, sigma 2 (logvar = log 4), eps 0.5: z = 1 + 2 x 0.5 = 2.0000
KL(N(1, 4) || N(0, 1)) by hand: 0.5 (sigma^2 + mu^2 - 1 - log sigma^2) = 0.5 (4 + 1 - 1 - 1.3863)
    = 1.3069; kl_normal: 1.3069
KL of N(0, 1) from itself: 0.0000
Actual output from this lesson's run.
```

What it means

z = 1 + 2 x 0.5 = 2.0000, and the KL divergence by hand and by kl_normal are both 1.3069. A standard normal is 0.0000 from itself: the KL term costs nothing only when a code cloud is exactly N(0, 1). The network works with the log-variance so that σ is always positive.

WHY CHOOSE THIS?

Both pieces have closed forms: no sampling needed for the KL.

WHY NOT THIS? (TRADEOFFS)

The closed form holds for Gaussians; other choices need estimates.

2. The KL formula against a Monte Carlo estimate

DEFINITION — start here, no prior knowledge assumed

The average of log q(z) – log p(z) over a million draws from q, for four Gaussians.

REAL-WORLD SCENARIO

Checking a formula for a dice's average by rolling it a million times.

ACTUAL OUTPUT

```
      mu  sigma   formula  Monte Carlo  standard error
     0.0    1.0    0.0000       0.0000          0.0000
     1.0    2.0    1.3069       1.3041          0.0029
    -2.0    0.5    2.3181       2.3190          0.0011
     0.5    0.1    1.9326       1.9338          0.0007
Actual output from this lesson's run.
```

What it means

Every estimate is within about one standard error of the formula (1.3041 against 1.3069, standard error 0.0029). Narrow clouds far from 0 cost the most (2.3181 for μ = –2, σ = 0.5): the KL term pushes codes towards 0 and spreads them out.

WHY CHOOSE THIS?

An independent check of the formula the VAE relies on.

WHY NOT THIS? (TRADEOFFS)

A million draws for four numbers: the formula is what training uses.

3. Two gradient estimators

DEFINITION — start here, no prior knowledge assumed

The gradient of $E[(z - 3)^2]$ with respect to μ and σ, whose exact value is $(2(\mu - 3), 2\sigma)$, estimated from one draw at a time, 100,000 times. The **score function** (REINFORCE) multiplies f(z) by the gradient of log q(z); the **reparameterized** estimator differentiates f through $z = \mu + \sigma\varepsilon$.

REAL-WORLD SCENARIO

Two ways to learn which way to aim: notice only whether each throw scored well, or see exactly how far and which way each one missed.

MATHEMATICS

score function: $f(z)\,\partial \log q(z)/\partial\mu = f(z)(z-\mu)/\sigma^2$; reparameterized: $f'(z)\,\partial z/\partial\mu = f'(z)$, $\partial z/\partial\sigma = \varepsilon$

IMPLEMENTATION

```
def score_grad(f, mu, sigma, rng, n):
    z = mu + sigma * rng.standard_normal(n)
    return f(z) * (z - mu) / sigma ** 2, f(z) * ((z - mu) ** 2 / sigma ** 3 - 1 / sigma)
def reparam_grad(df, mu, sigma, rng, n):
    eps = rng.standard_normal(n); z = mu + sigma * eps
    return df(z), df(z) * eps
```

ACTUAL OUTPUT

```
  f and where                              exact       estimator              mean      spread (std)
  (z - 3)^2 at mu 0, sigma 1         (-6.00, 2.00)  score function     (-5.92, 1.91)    (14.85, 28.07)
  (z - 3)^2 at mu 0, sigma 1         (-6.00, 2.00) reparameterized     (-5.99, 1.97)      (1.99, 6.58)
  (z - 3)^2 at mu 2.9, sigma 0.1     (-0.20, 0.20)  score function     (-0.20, 0.20)      (0.55, 1.18)
  (z - 3)^2 at mu 2.9, sigma 0.1     (-0.20, 0.20) reparameterized     (-0.20, 0.20)      (0.20, 0.35)
both are right on average; the score function's d/d mu estimates spread 7.4 and 2.8 times wider than the
    trick's in the two cases
Actual output from this lesson's run.
```

What it means

Both are unbiased: their means match the exact gradients ((-5.92, 1.91) and (-5.99, 1.97) against (–6, 2)). Their noise differs greatly: one score-function estimate of d/dμ spreads 14.85, a reparameterized one 1.99. The score function uses only the value f(z), not its slope, and must learn the direction from many draws.

WHY CHOOSE THIS?

The trick gives low-noise gradients whenever f can be differentiated.

WHY NOT THIS? (TRADEOFFS)

It needs a differentiable f and a distribution that can be written as $\mu + \sigma\varepsilon$.

4. Learning a Gaussian with each estimator

DEFINITION — start here, no prior knowledge assumed

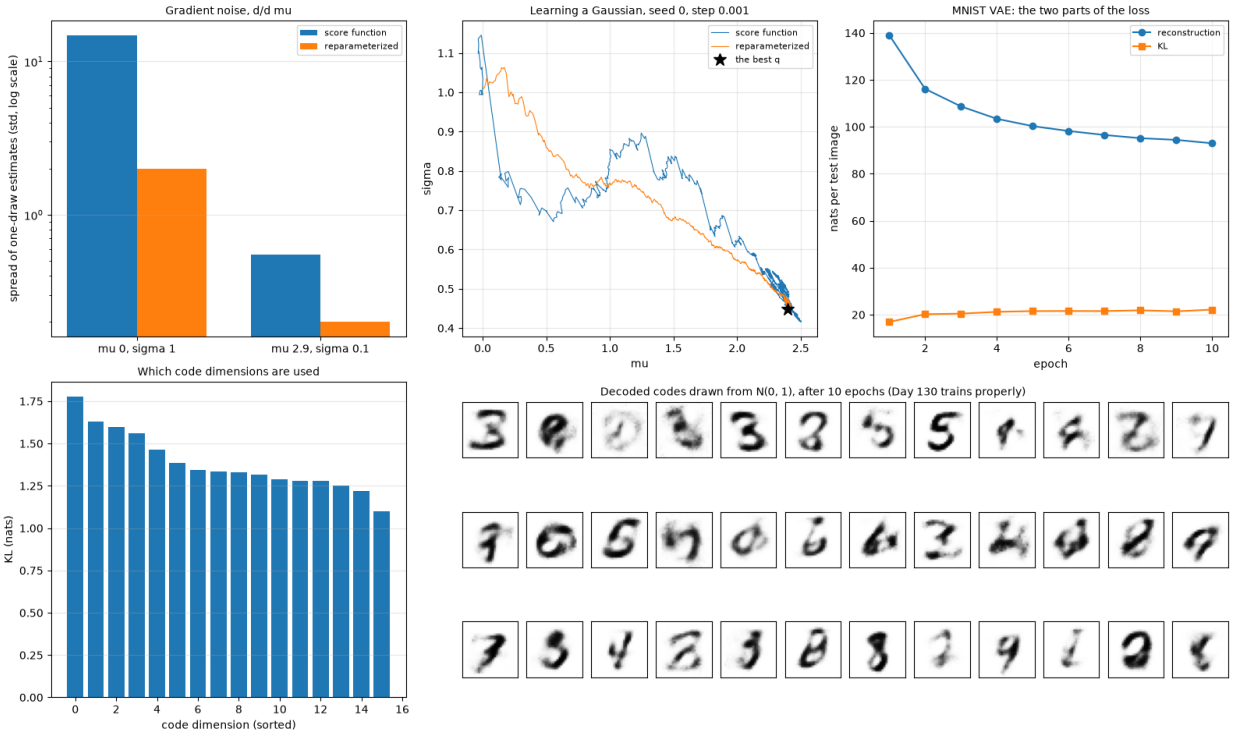
Find the Gaussian q minimising $E[(z-3)^2/(2 \times 0.25)] + KL(q \,||\, N(0, 1))$, one draw per step, 2,000 steps, 20 seeds, two step sizes. The best q is known exactly: the Bayesian posterior, $\mu = 2.4$ and $\sigma = \sqrt{0.2} \approx 0.447$.

REAL-WORLD SCENARIO

Two dart throwers improving by practice: one told only the score, one told where each dart landed.

ACTUAL OUTPUT

```
by hand, the best q is the Bayesian posterior: precision 1 + 1/0.25 = 5, so sigma = sqrt(0.2), mu
    = 3 x 4/5 = 2.4
  estimator        step size   blew up  mu (mean)  sigma (mean)    distance to the best
  score function        0.01   3 of 20      2.530         0.366          (0.169, 0.102)
  score function       0.001   0 of 20      2.405         0.450          (0.047, 0.036)
  reparameterized       0.01   0 of 20      2.419         0.435          (0.038, 0.033)
  reparameterized      0.001   0 of 20      2.404         0.448          (0.015, 0.010)
(the means and distances are over the runs that did not blow up)
Actual output from this lesson's run.
```



Top: noise of one-draw estimates; one seed's path towards the best q; the MNIST VAE's two loss terms. Bottom: KL per code dimension; codes drawn from N(0, 1) and decoded after 10 epochs.

What it means

At step size 0.01 the score-function version blew up in 3 of 20 runs, and those that survived ended further away ((0.169, 0.102)). The reparameterized version never blew up and landed closer at either step size ((0.038, 0.033), then (0.015, 0.010)). A smaller step rescues the score function ((0.047, 0.036)), but it is still about three times further away than the trick at the same step: noisier gradients force smaller, slower steps.

WHY CHOOSE THIS?

Learning with a known right answer.

WHY NOT THIS? (TRADEOFFS)

A one-dimensional problem; networks with many parameters were not tested here.

5. Correctness: a gradient check through a whole VAE

DEFINITION — start here, no prior knowledge assumed

Encoder 6-5-4 (two means, two log-variances), decoder 2-5-6, with ε held fixed; every weight nudged both ways.

REAL-WORLD SCENARIO

Freezing the wobble to check the aim's effect precisely.

IMPLEMENTATION

```
def vae_loss(enc, dec, X, rng, beta=1.0, acts_e=("relu", "linear"), acts_d=("relu", "linear"),
    eps=None):
    n = len(X); H = d126.forward(enc, X, list(acts_e)); out = H[-1]; k = out.shape[1] // 2
    mu, logvar = out[:, :k], out[:, k:]
    if eps is None:
        z, eps = reparam(mu, logvar, rng)
    else:
        z = mu + np.exp(0.5 * logvar) * eps
    A = d126.forward(dec, z, list(acts_d)); rec, dL = d126.recon_loss(A[-1], X, "bce")
    kl = float(kl_normal(mu, logvar).mean())
    gdec, dz = d126.backward(dec, A, list(acts_d), dL)
    dmu = dz + beta * mu / n
    dlogvar = dz * eps * 0.5 * np.exp(0.5 * logvar) + beta * 0.5 * (np.exp(logvar) - 1) / n
    genc, _ = d126.backward(enc, H, list(acts_e), np.hstack([dmu, dlogvar]))
    return rec + beta * kl, rec, kl, genc, gdec
```

ACTUAL OUTPUT

```
all 110 parameters: largest gap between backpropagation and finite differences 3.1e-09
with eps fixed, z is an ordinary function of the encoder's outputs, so ordinary backpropagation
    applies: that is the trick
Actual output from this lesson's run.
```

What it means

Backpropagation agrees with finite differences to 3.1e-09 over all 110 parameters. With ε fixed, the whole VAE is an ordinary deterministic function, so ordinary backpropagation applies: that is all the trick is. The gradient reaching z splits into $\partial z/\partial \mu = 1$ and $\partial z/\partial \log\sigma^2 = \varepsilon\sigma/2$, plus the KL term's own gradients.

WHY CHOOSE THIS?

The full VAE gradient, checked.

WHY NOT THIS? (TRADEOFFS)

The check holds ε fixed; training draws a new ε every step.

6. The trick in a real network

DEFINITION — start here, no prior knowledge assumed

A VAE on 20,000 MNIST images: 784 -> 256 -> 16 means and 16 log-variances; decoder 16 -> 256 -> 784; 10 epochs.

REAL-WORLD SCENARIO

A first rehearsal before the full performance on Day 130.

IMPLEMENTATION

```
def train_vae(X, k, hidden=256, epochs=10, batch=128, lr=1e-3, beta=1.0, seed=0, Xval=None):
    enc = d126.init_net([X.shape[1], hidden, 2 * k], seed); dec = d126.init_net([k, hidden,
        X.shape[1]], seed + 1)
    oe, od = d126.Adam(enc, lr), d126.Adam(dec, lr); rng = np.random.RandomState(seed + 2); hist =
        []
    for _ in range(epochs):
        o = rng.permutation(len(X))
        for s in range(0, len(X), batch):
            _, _, _, ge, gd = vae_loss(enc, dec, X[o[s:s + batch]], rng, beta)
            oe.step(enc, ge); od.step(dec, gd)
        ev = X[:2000] if Xval is None else Xval
        hist.append(vae_loss(enc, dec, ev, np.random.RandomState(99), beta)[:3])
    return enc, dec, hist
```

ACTUAL OUTPUT

```
   epoch  loss (negative ELBO)  reconstruction      KL   per test image, in nats
       1                155.95          139.04   16.91
       2                136.35          116.14   20.20
       3                129.14          108.69   20.45
       5                121.81          100.28   21.52
      10                115.14           92.98   22.16
trained in 26 s; KL per code dimension, largest first: [1.78, 1.63, 1.6, 1.56, 1.47, 1.38, 1.35, 1.34, 1.33,
    1.32, 1.29, 1.28, 1.28, 1.25, 1.22, 1.1]
dimensions carrying more than 0.1 nats: 16 of 16 (a dimension below that stays at the prior and carries
    nothing)
Actual output from this lesson's run.
```

What it means

The loss falls from 155.95 to 115.14 nats per test image in 26 seconds. The two parts move in opposite directions: reconstruction improves (139.04 to 92.98) while the KL rises (16.91 to 22.16), as the network learns to put more information into the codes and pays for it. All 16 dimensions carry information; the codes drawn from N(0, 1) already decode to digit-like shapes, some blurred.

WHY CHOOSE THIS?

Shows the two terms trading against each other.

WHY NOT THIS? (TRADEOFFS)

Ten epochs is a rehearsal; Day 130 trains for longer and judges the samples.

Interview preparation — technical questions

Q: What does a VAE's encoder output?

A: A mean and a log-variance per code dimension; the code is drawn from that Gaussian.

Q: What is the reparameterization trick?

A: $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim N(0, 1)$, so gradients flow through μ and σ.

Q: Why not just sample z?

A: Sampling is not differentiable; the alternative estimator was 7 times noisier (14.85 against 1.99).

Q: What is the VAE loss?

A: Reconstruction plus KL to N(0, 1): the negative ELBO, 115.14 nats after 10 epochs here.

Q: Why predict the log-variance?

A: Any real number is allowed, and $\sigma = \exp(\tfrac{1}{2} \log\text{-variance})$ stays positive.

Interview preparation — theoretical questions

Q: Why does the KL term matter?

A: It keeps code clouds near N(0, 1), so random codes decode sensibly (Day 130 measures this).

Q: What does the score-function estimator need?

A: Only values of f, not its gradient; the price is variance.

Q: Why does high variance hurt training?

A: It forces small steps: at step 0.01 the score function blew up in 3 of 20 runs.

Q: What does the ELBO bound?

A: The log-probability of the data: $\log p(x) \geq -(\text{reconstruction} + \text{KL})$.

Key takeaway and what's next

> **KEY TAKEAWAY**
> The reparameterization trick moves a VAE's randomness into a variable with no parameters, so the sampled code becomes an ordinary function of the encoder's outputs. Checked against Monte Carlo, finite differences and the score-function alternative, it gave unbiased gradients with far less noise. Tomorrow the same machinery is trained properly and its code space mapped.

- KL formula against a million draws: 1.3069 against 1.3041 (standard error 0.0029).
- Gradient noise, d/dμ: score function 14.85, reparameterized 1.99.
- Learning a Gaussian at step 0.01: score function blew up in 3 of 20 runs; the trick in none.
- VAE gradient check: 3.1e-09 over 110 parameters.

Suggested watch order, if starting from zero

- 1. The Reparameterization Trick (ML & DL Explained) — the trick's idea and why it is needed.
- 2. The Reparameterisation Trick | Variational Inference (Machine Learning and AI Academy) — the change of variables behind it, checked mathematically.

Day 130 preview

- **Training a VAE**, with a real latent-space visualisation and new digits generated from it.